Genetic algorithm for Traveling Salesman Problem

Haojie Xu
Wencheng Electric Power Supply Company, State Grid Corporation of China, China
812254330@qq.com

Yisu Ge*
College of Computer science and Artificial intelligence, Wenzhou University, China
ysg@wzu.edu.cn

Guodao Zhang
Department of Digital Media Technology, Hangzhou Dianzi University, China
guodaozhang@hdu.edu.cn

ABSTRACT

Traveling Salesman Problem (TSP) is one of the most famous NP-hard problems which is hard to find an optimal solution. Many heuristic algorithms are applied to find a suboptimal solution in a limited time. In this paper, we employ a Genetic Algorithm (GA) to solve the TSP, and a further study is conducted by evaluating the performance of different crossover and mutation methods with a heuristic strategy. Four experiments with different parameters are designed, which apply instances from benchmark TSPLIB. Partial-mapped crossover and rotate mutation with offspring-parent competition strategy has shown efficient gets the best results.

CCS CONCEPTS

• **Computing methodologies**; • **Artificial intelligence**; • **Search methodologies**; • **Discrete space search**;

KEYWORDS

Genetic algorithm, Traveling salesman problem, Artificial intelligence

ACM Reference Format:
Haojie Xu, Yisu Ge, and Guodao Zhang. 2022. Genetic algorithm for Traveling Salesman Problem. In *2022 The 5th International Conference on Computational Intelligence and Intelligent Systems (CIIS) (CIIS 2022), November 04–06, 2022, Quzhou, China.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3581792.3581798

1 INTRODUCTION

Traveling Salesman Problem (TSP) is one of the fundamental problems in computational intelligence and operation research, which is an NP-hard problem in combinatorial optimization. In TSP, there is a set of cities $C = \{c_1, c_2, c_3, ..., c_n\}$, and the distances between each pair of cities are given. The traveler should find the shortest possible route that visits each city exactly once and returns to the original one. The problem seems simple but has a high degree of algorithm complexity, of which the amount of calculation will be rapidly improved with the increase of city number. The TSP is similar to many other puzzles [1], such as multimodal optimization problems [2, 3], electric vehicle charging scheduling [4], warehouse stacker cranes scheduling, supply chain management [5], transport route optimization [6], and machine scheduling to drill holes in a circuit board [7].

To accelerate the speed of TSP solving, a great number of methods are proposed. The branch and bound method, linear programming, and dynamic programming are studied to get the exact solution. But figuring out the optimal solution to the TSP is sophisticated, and finding the suboptimal solution might be useful as well in practical application. Therefore, numerous heuristic algorithms are put forward to find a suboptimal solution in a limited time, include: simulate anneal arithmetic (SA) [8], tabu search (TS) [9], artificial neural network (ANN) [10], particle swarm optimization (PSO) [11], ant colony optimization (ACO) [12] and genetic algorithm (GA) [13]. The heuristic algorithms are inspired by the natural phenomena, which are guided by the heuristic information for faster convergence, but may fall into the local optimal solution. In this paper, a study about GA for TSP is conducted with different crossover, mutation, and heuristic strategies.

Genetic algorithm(GA) [13] is a meta heuristic algorithm for generating high-quality solutions to optimization and search problems [14], which is inspired by the process of natural selection mechanism. It is motivated by the hope that good parents can generate better offspring. GA code a solution as a chromosome, and a group of chromosomes compose a population [15]. The performance of chromosomes is evaluated by the fitness function, and the selection process is applied to obtain better solutions for the problem. Then the biological processes such as crossover and mutation [16] are utilized for generating new offspring, and a new population is generated. The processes of gene generating and selection are repeated generation by generation until certain conditions are fulfilled. The best individual is outputted as the final solution. Figure 1 shows the procedure of GA. The *Gen* is the given parameter about iteration times, *G* means the current iteration times, which will increase after the new population generating, and GA procedure will be terminated when $G > Gen$.

The crossover and mutation with heuristic strategies are the vital parts of the GA, which directly affect the convergence speed of the algorithm. Therefore, partial-mapped crossover (PMX) and order crossover (OX) methods, rotate and exchange mutation approaches, and offspring-parents competition strategy are considered in this paper.

The remaining part of the paper is organized as follows. Section II reviews the related works of GA for TSP. Then, in Section III, the details of GA for TSP are introduced, following which is the verification of the effectiveness and efficiency of each operation in GA by extensive experiments in Section IV. Finally, conclusions are given in Section V.

*Corresponding Author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIIS 2022, November 04–06, 2022, Quzhou, China
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9761-2/22/11…$15.00
https://doi.org/10.1145/3581792.3581798

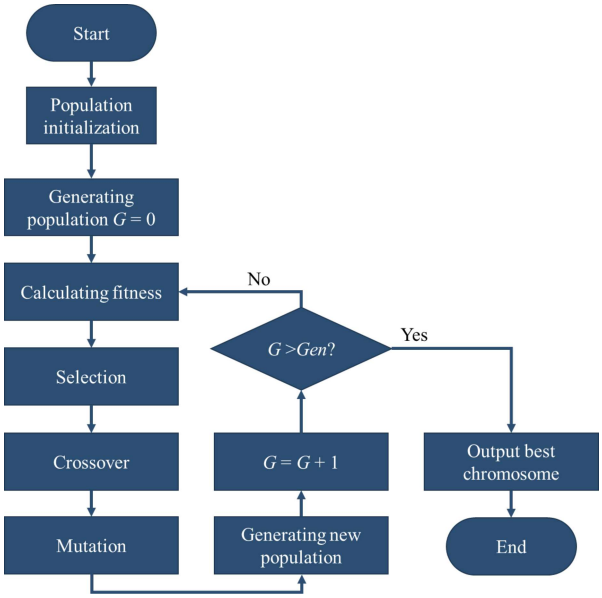


Figure 1: Procedure of GA

2 RELATED WORKS

GA is a method of evolutionary computation, which was proposed by J. Holland [13] in 1975 and inspired by Darwin's theory of evolution. Then, K. A. De Jong built the frameworks of GA and gave some guidance on parameter selection [17]. Since then, GA had attracted more and more attention. In 1989, D. J. Goldberg published the monograph "Genetic Algorithms in Search, Optimization and Machine Learning" [18] which summarizes the main research results on GA. After that, the GA had ushered in a period of rapid progress, and was widely applied in practical applications after "Handbook of Genetic Algorithms" [19] was published. GA had obtained great progress in many aspects through several decades of development, and there are plenty of works focusing on the TSP.

For solving the TSP, various works were designed based on GA, and the potential of GA was further studied. H. Sengoku [20] proposed a hybrid algorithm using GA and heuristics for the rapid solution of TSP, and the 2opt mutation and greedy sub tour crossover were utilized. For comparing the performance of SA, GA, and ACO, H. H. Mukhairez [21] conducted a comparison study to evaluate the algorithm performance, and GA balanced the execution time and shortest distance of TSP. S. A. Haroun [22] further compared the performance of GA and ACO, and conducted a conclusion that GA is excellent with small instances, and maintains a steady improvement behavior. A. Gharib [23] presented the comparison of PSO and GA, and GA performs better. N. M. Razali [24] compared different parent selection strategies in GA for TSP, and the results show that tournament selection strategy outperforms other methods in low computing times, but the rank-based roulette wheel selection is better when the problem size increases. Parameter values are important in GA, and A. Rexhepi [25] analyzed the impact of parameters on GA for TSP, and the relationship between the size of initial population, mutation probability, and the number of generations are revealed.

Plenty of research was described for optimizing the operations of GA. F. Yu [26] presented an improved selection method based on fitness proportionate selection, which is better than roulette wheel selection. Y. Deng [27] paid attention to the initial population strategy to improve the performance of GA for TSP. Z. Tao [28] raised the Extend PMX to promote the GA for TSP by modifying the crossover method. Similarly, S. Akter [29] presented a crossover operator consisting of two crossover point selections and new offspring creation by performing the cost comparison. For jumping out of the local optimal solution, Y. Tsujimura [30] introduced a measurement of population diversity using the concept of information entropy. To optimize large-scale distribution networks, Y. Sakurai [31] developed a multi-world intelligent GA that combines the high-speed GA with an intelligent GA holding problem-oriented knowledge. H. D. Nguyen [32] advanced a method based on a parallel implementation of a multi-population steady-state GA with local search heuristics, to get the high-quality solution of large-scale TSP. For tracking the multiple peak problem, a development version of GA is put forward and named niche algorithm. T. Huang [33] proposed a niching memetic algorithm to enable the parallel search of multiple optimal solutions in TSP.

The research of GA for TSP has been greatly developed in recent decades, but more details about the methods should be further explored. In this paper, the PMX and OX crossover methods with two different mutations and a heuristic strategy are compared in the experiments. And four experiments with different parameters are projected for better evaluating the performance of each operation.

3 METHOD

In this paper, different cities in TSP are coded by different genes with different numbers, and each individual in the population represents a possible solution to the TSP. For example, as presented in Figure 2, there is a sample of 5 cities TSP, including Beijing, Shanghai, Singapore, Seoul, and Tokyo, and the position of each city is already given. For converting the TSP to the GA, cities are coded, and the number 1 to 5 denotes the cities Beijing, Shanghai, Singapore, Seoul, and Tokyo, respectively. The sequence of (1, 2, 3, 4, 5) means the salesman travels from Beijing, and goes through Shanghai, Singapore, Seoul, and Tokyo step by step, and goes back to Beijing finally.

3.1 Fitness Calculation and Selection

Chromosome fitness evaluation is the most important part of GA, and fitness function is applied to evaluate the quality of individuals for chromosome selection. The most direct way to evaluate the chromosomes of TSP is calculating the distance of travel. So, the Euclidean distance is employed to evaluate the performance of chromosomes. But it is not suitable for fitness calculation, therefore, the reciprocal of distance is used, as presented in Eq. (1)~(3).

$$Fit\,(x) = \frac{1}{dis_{total}\,(x)} \tag{1}$$

$$dis_{total}\,(x) = dis\,(c_n, c_0) + \sum_{i=0}^{n-1} dis\,(c_i, c_{i+1}) \tag{2}$$

$$dis\,(c_i, c_j) = \sqrt{(c_i \cdot x - c_j \cdot x)^2 + (c_i \cdot y - c_j \cdot y)^2} \tag{3}$$

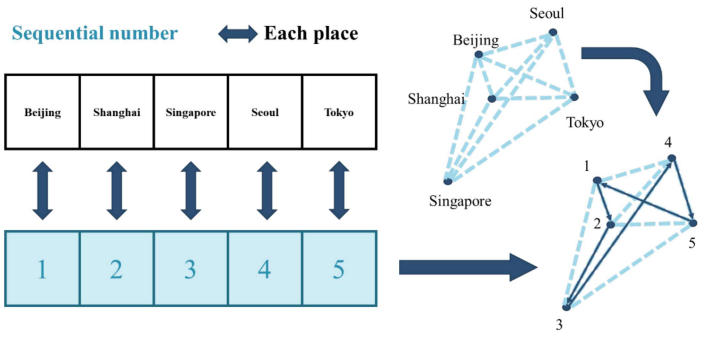


Figure 2: Illustration of Chromosome Coding

Where x is a solution in TSP, c_i and c_j are two cities in travel, of which the positions are $(c_i.x, c_i.y)$ and $(c_j.x, c_j.y)$, respectively. Directly using the original city position of TSP may make the value of travel distance too big, and the fitness might be too small for competition because small numbers might be ignored in computation. Therefore, the position of the city should be normalized before the distance calculation.

After fitness evaluation, $p_r \times Popsize$ individuals are selected based on the roulette wheel selection, such that the higher the fitness, the higher the opportunity for the individual to be selected. p_r means reproduction probability of GA, and $Popsize$ is the population size. The selection probability of each chromosome is $P(x_i)$:

$$P(x_k) = \frac{Fit(x_k)}{\sum_{j=1}^{Popsize} Fit(x_j)} \tag{4}$$

3.2 Crossover

Crossover is one of the important methods for new chromosome generating, which imitates gene crossover in biology. Two chromosomes are selected at first, and part of the genes in two chromosomes are exchanged for creating the new offspring if $p > p_c$. p is a random number uniformly distributed in [0, 1), and p_c denotes the crossover probability which is a predefined parameter.

There are many crossover methods in GA, but various approaches are unsuitable for TSP, because the crossover may produce the invalid solution of TSP which is not an unrepeated array. In this paper, two different crossover methods for TSP are applied and named Partial-Mapped Crossover(PMX) [34] and Order Crossover(OX) [35], respectively.

PMX is designed for TSP, which checks and corrects the offspring after gene exchanging. PMX includes 5 steps, the simple illustration is shown in Figure 3

- Step 1: Select two chromosomes in the population randomly.
- Step 2: Select two positions for crossing arbitrarily.
- Step 3: Exchange the genes between the two positions, and two offspring are generated.
- Step 4: Find the position mapping rules by the match region.
- Step 5: Check the offspring, and correct the repeat number by position mapping rules.

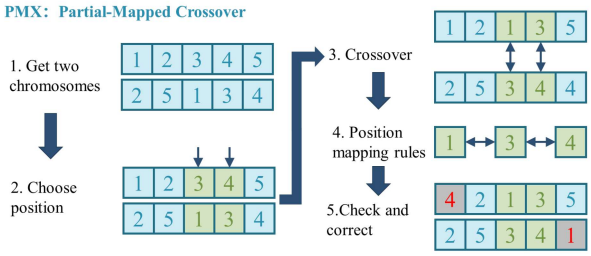


Figure 3: Illustration of PMX

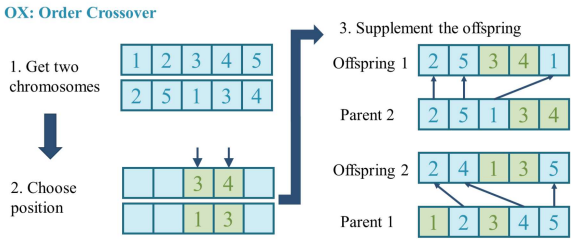


Figure 4: Illustration of OX

As presented in Figure 3, an example of PMX is shown. Two chromosomes are selected from the population, (1, 2, 3, 4, 5) and (2, 5, 1, 3, 4). The two positions in the chromosome are randomly selected for crossing, the example chooses the third to fourth genes. Then, exchange the genes between the two positions and keep the rest of the genes, two invalid offspring are generated, (1, 2, 1, 3, 5) and (2, 5, 3, 4, 4). The exchanged genes compose the position mapping rules. In the example, (1, 3) is exchanged with (3, 4), so the 1 corresponding to 3, and 3 is mapped to 4. Finally, check each unchanged gene in offspring, whether it is repeated with the exchanged genes. If the gene is repeated, correct the gene by the position mapping rules. Check each unchanged gene until the end of the chromosome, and two offspring are gotten, (4, 2, 1, 3, 5) and (2, 5, 3, 4, 1).

Different from PMX, OX is simpler and without the check step, which contains 4 steps, as shown in Figure 4

- Step 1: Randomly select two chromosomes for parents at first.

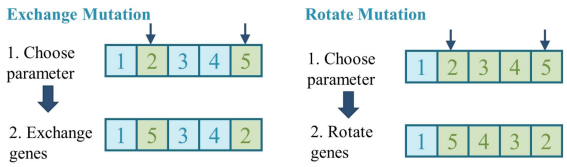


Figure 5: Contract of rotate mutation and exchange mutation

- Step 2: Select two positions for crossing arbitrarily, and generate two offspring.
- Step 3: Find the changed genes in the other parent chromosome, and put the unchanged genes to the new offspring orderly.

An example is shown in Figure 4. First, two chromosomes are chosen from the population, (1, 2, 3, 4, 5) and (2, 5, 1, 3, 4). Then two positions for crossover are chosen, the genes between the positions are copied to the new offspring. Then, find the copied genes in the other parent. Finally, fill up the block of offspring by the no copied genes from the other parent orderly, and two offspring are generated, (2, 4, 1, 3, 5) and (1, 2, 3, 4, 5).

3.3 Mutation

The Mutation is exerted to create new genes. A chromosome is selected in the population, and the mutation is operated if $p > p_m$. pm represents the predefined mutation probability. In an unrepeated sequence, the most direct way of mutation is exchanging two genes in the chromosome, but the convergence speed of GA might be slower. Therefore, the rotate mutation is proposed, which includes 2 steps:

- Step 1: Randomly choose two positions for genes rotation.
- Step 2: Rotate the genes between the two positions.

The contract of exchange mutation and rotate mutation is illustrated in Figure 5.

Exchange mutation only changes two genes, but the rotate mutation rotates the genes' order between two positions, which is much efficient for new genes generating. The above two mutations are two kinds of 2opt mutations, rotate mutation changes the edge of the trajectory and exchange mutation changes the point of the traveling.

3.4 Other Heuristic Strategy

For faster convergence, the offspring are competing with parents after the offspring generating (crossover and mutation), and only the better ones can be kept. The offspring-parent competition strategy is used to avoid the bad genes affecting heredity, which speeds up the optimal solution finding. The process is illustrated in Figure 6.

Furthermore, the elitist selection strategy is utilized for keeping the best individual. The best and worst solutions will be found in each generation, and if the best individual in this generation is worse than the historical best, the worse individual will be replaced by the historical best. This operation guarantees that the solution quality obtained by the GA will not decrease from one generation to the next [36].

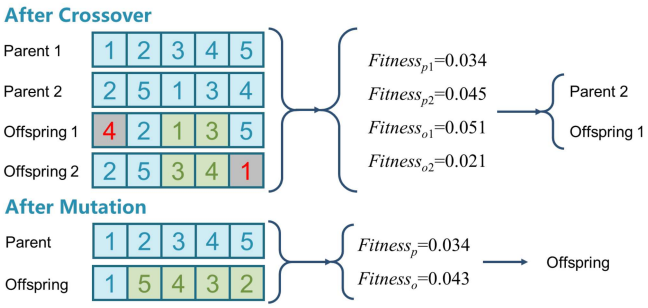


Figure 6: Illustration of Competition between Parents and Offspring

Table 1: Parameter Setting in Experiments

Variable	parameter 1	parameter 2	Parameter 3
Popsize	200	2000	200
Gen	5000	50000	5000
p_r	0.5	0.5	0.5
p_c	0.8	0.8	0.8
p_m	0.08	0.08	0.08
RunTime	5	5	20

4 COMPUTATIONAL RESULT

For evaluating the performance of each operation in GA, four experiments are designed. The experiments are implemented in C++ and run on an Intel Core i5-1135G7 processor with a 2.42GHz CPU and 16GB RAM. The trajectory images are drawn by OpenCV. And the parameter setting of *Popsize*, *Gen*, p_r, p_c, and p_m are listed in Table 1.

Three different groups of experimental parameters are set, which are used to prove the performance of the method. The algorithm is tested on TSPLIB which is a library of sample instances for TSP, and six TSP instances are implemented in the experiments, including eil101, lin105, tsp225, a280, and pr1002. The values of city position are normalized to [0, 200] before the GA starts.

4.1 Experiment 1: Influence of Parameter pc and pm

The probability of crossover p_c and mutation p_m play vital roles in GA, and the selection of p_c and p_m may greatly affect the results. Therefore, a preliminary experiment is designed for evaluating the influence of p_c and p_m parameter selection. The range of p_c is 0.1~1.0 with 0.1 intervals, and the range of p_m is 0.01~0.20 with 0.01 intervals in the experiment. The TSP instance eil101 is applied of which the number of cities is 101. PMX and rotate mutation with the offspring-parent competition are utilized in the crossover. The result is displayed in Figure 7.

As shown in Figure 7, the variations of fitness are small and irregular, which means the fitness of GA is not sensitive to the changes in p_c and p_m. Therefore, the parameter setting with the best performance is chosen for the remaining experiments, of which the p_c is 0.8 and p_m is 0.08.

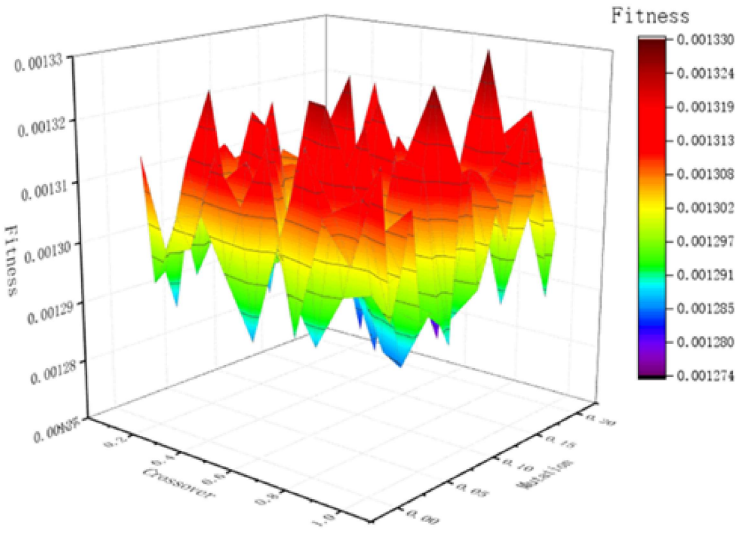


Figure 7: The Influence of pc and pm on Fitness

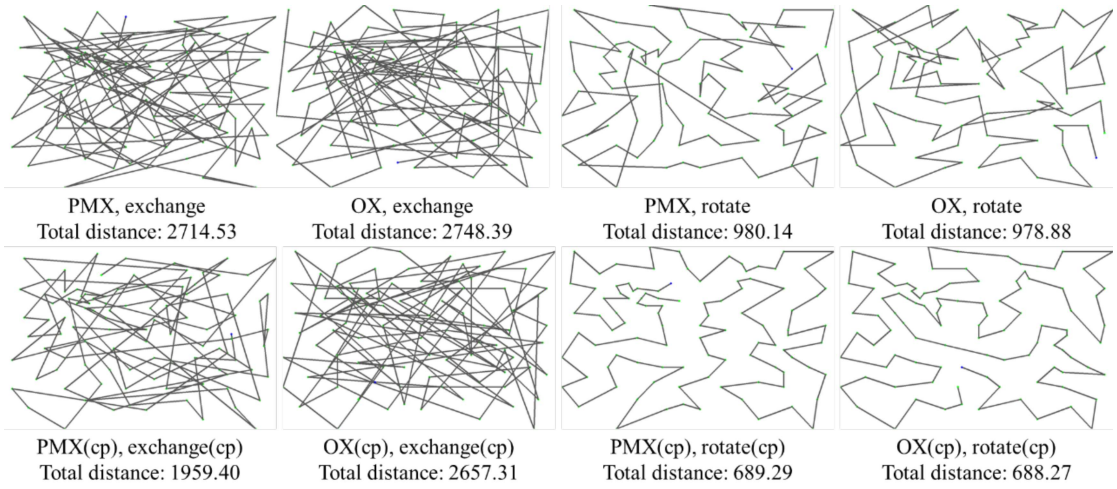


Figure 8: Result Different on Crossover and Mutation with a Strategy

4.2 Experiment 2: Performance of Different Crossover and Mutation

We carried out the second set of experiments to assess the contributions of different crossover and mutation operations. The instance eil101 with parameter 1 in Table 1 is applied. PMX and OX operations are compared, and exchange mutation competes with the rotate mutation. In addition, the influence of offspring-parent competition strategy is evaluated as well.

In Figure 8, trajectories of travel are depicted, the images in the first row are obtained by the methods directly using the offspring to instead parents after crossover and mutation. The images in the second row are gotten by the methods with the offspring-parent competition. The first column images are generated from the method using PMX with exchange mutation; the second column images are produced from the GA using OX with exchange mutation; the third column images are picked up by the method using PMX with rotate mutation; the last column images are come by the GA using OX with rotate mutation.

As shown in Figure 8, the chaos of the sub-image reflects the performance of the results, the more regular the better result is. The trajectories obtained by rotate mutation are less overlapping, and the total distance of travel is shorter as well. It means the rotate mutation greatly improves the GA performance and is directly reflected in the images. Comparing the results of whether the offspring-parent competition strategy is used, we can see the methods with offspring-parent competition perform better than the methods directly instead of parents by offspring. The heuristic

Table 2: Results in Different Instances on TSPLIB

Instance	eil101	lin105	tsp225	a280	pr1002
City number	101	105	225	280	1002
Distance(PMX +p1)	689.29	17422.40	4674.85	3193.17	1997905.19
Distance(PMX+p2)	691.35	17865.65	4652.68	2987.04	300757.47
Distance(OX+p1)	688.27	18158.94	5759.84	4981.48	2093843.89
Distance(OX+p2)	693.26	17599.82	4534.59	3150.83	313767.14
Distance(GT)	688.19	14383.00	3859.00	3632.84	259066.66

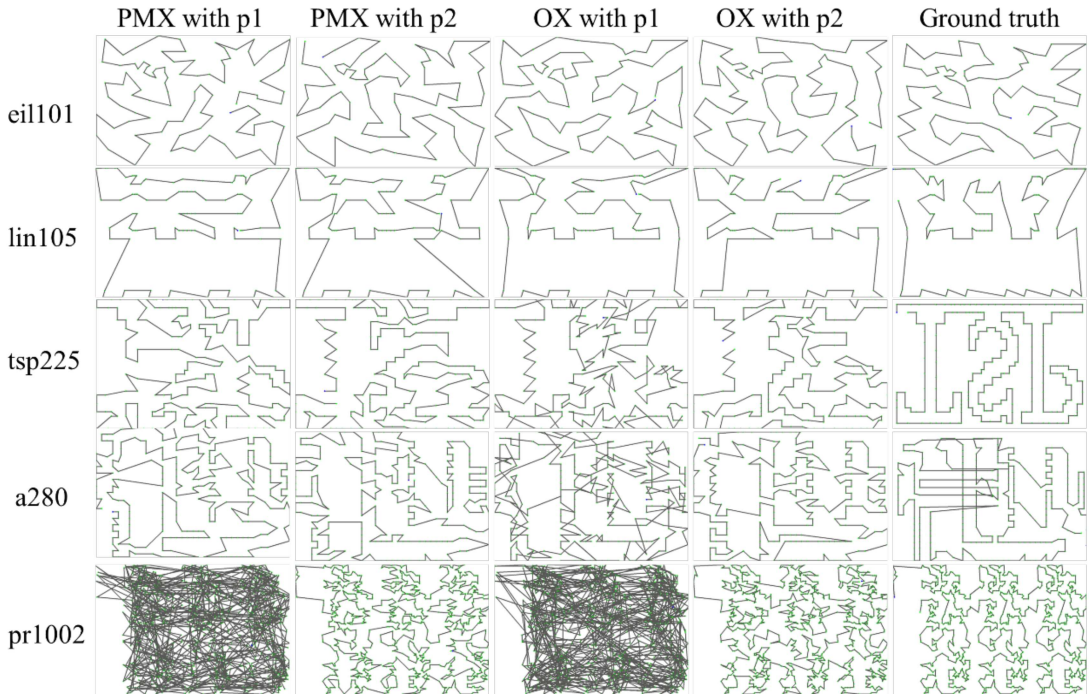


Figure 9: Results in Different Instances on TSPLIB

strategy can help the GA to alleviate the defects from crossover and mutation. The PMX results outperform the results of OX, except the result of OX with rotate mutation and offspring-parent competition. Most of the performances from the two crossovers are similar, except the result of exchange mutation and offspring-parent competition. So, it is pretty hard to decide which method is better, and further experiments are designed.

4.3 Experiment 3: Testing Crossovers with Different Instances and Parameters

GA is tested on TSPLIB with different parameters, and five instances are employed in this part. The result is presented in Table 2 and Figure 9. Table 2 lists the average best result, and the trajectories of travel are given in Figure 9.

In Table 2 and Figure 9, p1 means parameter 1 in Table 1 is applied, and p2 is parameter 2. Distance (PMX+p1) is the travel distance obtained by GA with PMX and parameter 1; Distance (PMX+p2) is gotten by GA with PMX and parameter 2. Similarly, the Distance (OX+p1) and Distance (OX+p2) means GA with OX and parameters 1 and 2 respectively. Distance (GT) is acquired from the official ground truth.

As shown in Table 2 and Figure 9, the result of PMX and OX are similar as well. The GA can find the approximate solution with parameter 1 while the city number is around 100 and 200, and OX seems better. But in pr1002, the GA with parameter 1 performs worse than the GA with parameter 2, and PMX shows great potential. It means that while the city number increases to 1000, the parameter *Popsize* and *Gen* should be enlarged as well, and the different crossover may be suitable for different sizes of problems. The ground truth in a280 is worse than the result of PMX and OX, the trajectories present the reason that there are several crossed travel paths in the ground truth.

Table 3: Average and Standard Deviation of PMX and OX Result

		PMX	OX
eil101	M	696.23	695.56
	Σ	11.59	14.00
lin105	M	17396.12	17395.30
	Σ	622.05	764.00
tsp225	M	4604.99	6067.92
	Σ	94.49	108.51
a280	M	3184.71	5057.64
	Σ	88.30	115.28
pr1002	M	1981467.14	2138981.49
	Σ	37253.96	46355.05

4.4 Experiment 4: Stability of PMX and OX

For accurate evaluation of the different crossover performances, experiment 4 is projected. Parameter 3 in Table 1 is used for evaluating the stability of the PMX and OX, the results are presented in Table 3. Where μ represents the average travel distance, σ is the standard deviation.

In Table 3, the average and standard deviation of travel distance are listed. In the TSP with a small number of cites (eil101 and lin105), average of OX gets the better results which even outperforms the ground truth. But the standard deviation of OX is bigger, which means the distribution of results is wider and the stability of the algorithm is weaker. With the growth of the city number, the PMX presents great potential and is better when the city amount is over 200.

5 CONCLUSION AND FUTURE WORK

In this paper, GA is employed to find a suboptimal solution for the case of time-constrained. PMX and rotate crossover with offspring-parent competition strategy are applied to figure out the TSP. PMX is used to keep the potential genes subsequences, rotate crossover focuses on the new genes generating, and offspring-parent competition strategy is applied to select better chromosomes for faster convergence. In the experiments, various crossover and mutation methods with different parameters are evaluated in benchmark TSPLIB. PMX and rotate crossover with the offspring-parent competition get the best result in experiments, and OX may get better solution in small size of problem.

The crossover approaches in GA may break the existing dominant genes, the crossing genes are only selected in the middle of the sequence and ignored the genes in the sides. Therefore, how to maintain the dominant genes in the crossover with uniform gene selection is further considered.

ACKNOWLEDGMENTS

This work is supported by the Wenzhou Science and Technology Plan Project (G20220035), and General Scientific Research Fund of Zhejiang Provincial Education Department (Y202248776).

REFERENCES

[1] Akter, S., N. Nahar, M. ShahadatHossain and K. Andersson (2019). A new crossover technique to improve genetic algorithm and its application to TSP. 2019 International Conference on Electrical, Computer and Communication Engineering (ECCE), IEEE.
[2] Alhanjouri, M. A. and B. Alfarra (2013). "Ant colony versus genetic algorithm based on travelling salesman problem." Int. J. Comput. Tech. Appl 2(3): 570-578.
[3] Baluja, S. and R. Caruana (1995). Removing the genetics from the standard genetic algorithm. Machine Learning Proceedings 1995, Elsevier: 38-46.
[4] Chen, Z.-G., Z.-H. Zhan, H. Wang and J. Zhang (2019). "Distributed individuals for multiple peaks: A novel differential evolution for multimodal optimization problems." IEEE Transactions on Evolutionary Computation 24(4): 708-719.
[5] Davis, L. (1985). Applying adaptive algorithms to epistatic domains. IJCAI.
[6] Davis, L. (1991). Handbook of genetic algorithms.
[7] De Jong, K. A. (1975). An analysis of the behavior of a class of genetic adaptive systems, University of Michigan.
[8] Deneubourg, J.-L., J. M. Pasteels and J.-C. Verhaeghe (1983). "Probabilistic behaviour in ants: a strategy of errors?" Journal of theoretical Biology 105(2): 259-271.
[9] Deng, Y., Y. Liu and D. Zhou (2015). "An Improved Genetic Algorithm with Initial Population Strategy for Symmetric TSP." Mathematical Problems in Engineering 2015: 212794.
[10] Gharib, A., J. Benhra and M. Chaouqi (2015). "A performance comparison of PSO and GA applied to TSP." International Journal of Computer Applications 130(15): 34-39.
[11] Glover, F. (1986). "Future paths for integer programming and links to artificial intelligence." Computers & operations research 13(5): 533-549.
[12] Goldberg, D. E. and J. H. Holland (1988). "Genetic algorithms and machine learning." Machine Learning 3(2-3): 95-99.
[13] Gorges-Schleuter, M. (1989). ASPARAGOS an asynchronous parallel genetic optimization strategy. Proc. 3rd ICGA.
[14] Hacizade, U. and I. Kaya (2018). "Ga based traveling salesman problem solution and its application to transport routes optimization." IFAC-PapersOnLine 51(30): 620-625.
[15] Haroun, S. A. and B. Jamal (2015). "A performance comparison of GA and ACO applied to TSP." International Journal of Computer Applications 117(19): 28-35.
[16] Holland, J. (1975). Adaptation in natural and artificial systems : an introductory analysis with application to biology, control and artificial Intelligence.
[17] Huang, T., Y.-J. Gong, W.-N. Chen, H. Wang and J. Zhang (2020). "A probabilistic niching evolutionary computation framework based on binary space partitioning." IEEE transactions on cybernetics.
[18] Huang, T., Y.-J. Gong, S. Kwong, H. Wang and J. Zhang (2019). "A niching memetic algorithm for multi-solution traveling salesman problem." IEEE Transactions on Evolutionary Computation 24(3): 508-522.
[19] Katoch, S., S. S. Chauhan and V. Kumar (2021). "A review on genetic algorithm: past, present, and future." Multimedia Tools and Applications 80(5): 8091-8126.
[20] Kennedy, J. and R. Eberhart (1995). Particle swarm optimization. Proceedings of ICNN'95-international conference on neural networks, IEEE.
[21] Li, J.-Y., Z.-H. Zhan, H. Wang and J. Zhang (2020). "Data-driven evolutionary algorithm with perturbation-based ensemble surrogates." IEEE Transactions on Cybernetics.
[22] Liu, W.-L., Y.-J. Gong, W.-N. Chen, Z. Liu, H. Wang and J. Zhang (2019). "Coordinated charging scheduling of electric vehicles: A mixed-variable differential evolution approach." IEEE Transactions on Intelligent Transportation Systems 21(12): 5094-5109.
[23] McCulloch, W. and W. Pitts (1943). "A logical Calculus of Ideas Immanent in Nervous Activity." The Bulletin of Mathematical Biophysics 5: 115-133.
[24] Mitchell, M. (1998). An introduction to genetic algorithms, MIT press.
[25] Mukhairez, H. H. and A. Y. Maghari (2015). "Performance comparison of simulated annealing, GA and ACO applied to TSP." International Journal of Intelligent Computing Research (IJICR) 6(4): 647-654.
[26] Nguyen, H. D., I. Yoshihara, K. Yamamori and M. Yasunaga (2007). "Implementation of an effective hybrid GA for large-scale traveling salesman problems." IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics) 37(1): 92-99.
[27] Rao, T. S. (2018). "An Evaluation of ACO and GA TSP in a Supply Chain Network." Materials Today: Proceedings 5(11): 25350-25357.
[28] Razali, N. M. and J. Geraghty (2011). Genetic algorithm performance with different selection strategies in solving TSP. Proceedings of the world congress on engineering, International Association of Engineers Hong Kong.
[29] Rexhepi, A., A. Maxhuni and A. Dika (2013). "Analysis of the impact of parameters values on the Genetic Algorithm for TSP." International Journal of Computer Science Issues (IJCSI) 10(1): 158-164.
[30] Sakurai, Y., T. Onoyama, S. Kubota, Y. Nakamura and S. Tsuruta (2006). A multi-world intelligent genetic algorithm to interactively optimize large-scale TSP. 2006 IEEE International Conference on Information Reuse & Integration, IEEE.
[31] Sengoku, H. and I. Yoshihara (1998). A fast TSP solver using GA on JAVA. Third International Symposium on Artificial Life, and Robotics (AROB III'98).
[32] Steinbrunn, M., G. Moerkotte and A. Kemper (1997). "Heuristic and randomized optimization for the join ordering problem." The VLDB Journal 6(3): 191-208.

[33] Tao, Z. (2008). TSP Problem solution based on improved Genetic Algorithm. 2008 Fourth International Conference on Natural Computation, IEEE.
[34] Tsujimura, Y. and M. Gen (1998). Entropy-based genetic algorithm for solving TSP. 1998 Second International Conference. Knowledge-Based Intelligent Electronic Systems. Proceedings KES'98 (Cat. No. 98EX111), IEEE.
[35] Yu, F., X. Fu, H. Li and G. Dong (2016). Improved roulette wheel selection-based genetic algorithm for TSP. 2016 International conference on network and information systems for computers (ICNISC), IEEE.
[36] Zhou, Z.-w. and T.-m. Ding (2007). "Research on holes machining path planning optimization with tsp and ga." Modular Machine Tool & Automatic Manufacturing Technique 66(7): 30-32.